Project - High Level Design

on

Multimodal

Hospitality

Creator

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Tanish Israni	EN22IT301112
2	Samarth Patidar	EN22IT301087
3	Yash Dashore	EN22CS3011100
4	Ramakant Singh	EN22IT301077
5	Abhishek Gupta	EN22EL301004
6	Sourabh Patel	EN22IT301106

Group Name: Group 10 D12

Project Number: GAI-48

Industry Mentor Name: Mr. Yash Bansal

University Mentor Name: Ajaj Khan

Academic Year: 2026

Table of Contents

1. Introduction.
   1.1. Scope of the document :
   This document describes the high-level design of a Streamlit-based multimodal Generative AI demo for hospitality concept visualization. The system accepts a user prompt and generates both descriptive text and a concept image using external AI APIs.

   1.2. Intended Audience :
   Faculty or evaluators reviewing the architecture
   Project team members preparing presentation or report material
   Developers who want to understand the implementation
   Stakeholders interested in the solution approach

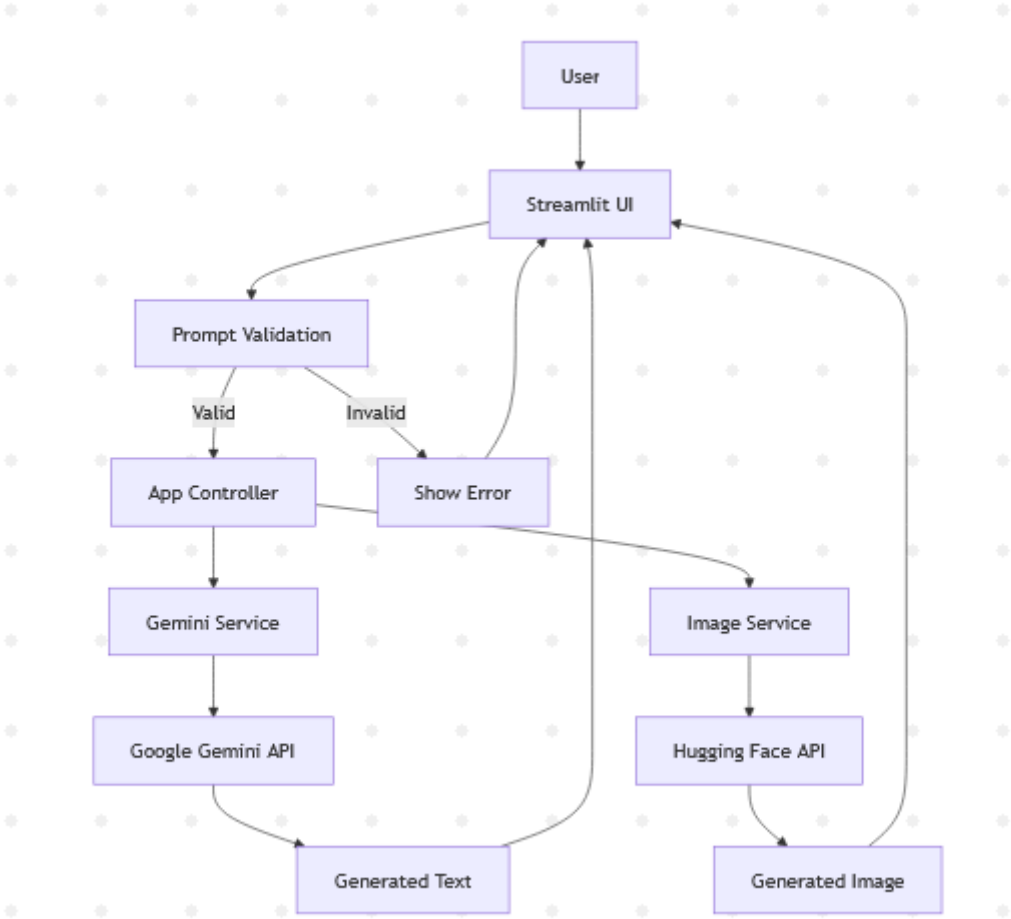
   1.3. System overview :
   The system is a lightweight web application that runs locally on Streamlit. It integrates:
   1. Google Gemini API for text generation
   2. Hugging Face Stable Diffusion inference API for image generation

   The application is designed as a demo-oriented proof of concept focused on simple user interaction and clear multimodal output.

2. System Design.
   2.1. Application Design :
   The application uses a simple single-tier app structure:

   1. Presentation layer: Streamlit UI in `app.py`
   2. Service layer: Gemini and Hugging Face API integrations in dedicated service files
   3. Shared configuration and validation utilities in helper modules
   4. External services: Gemini API and Hugging Face model inference endpoint

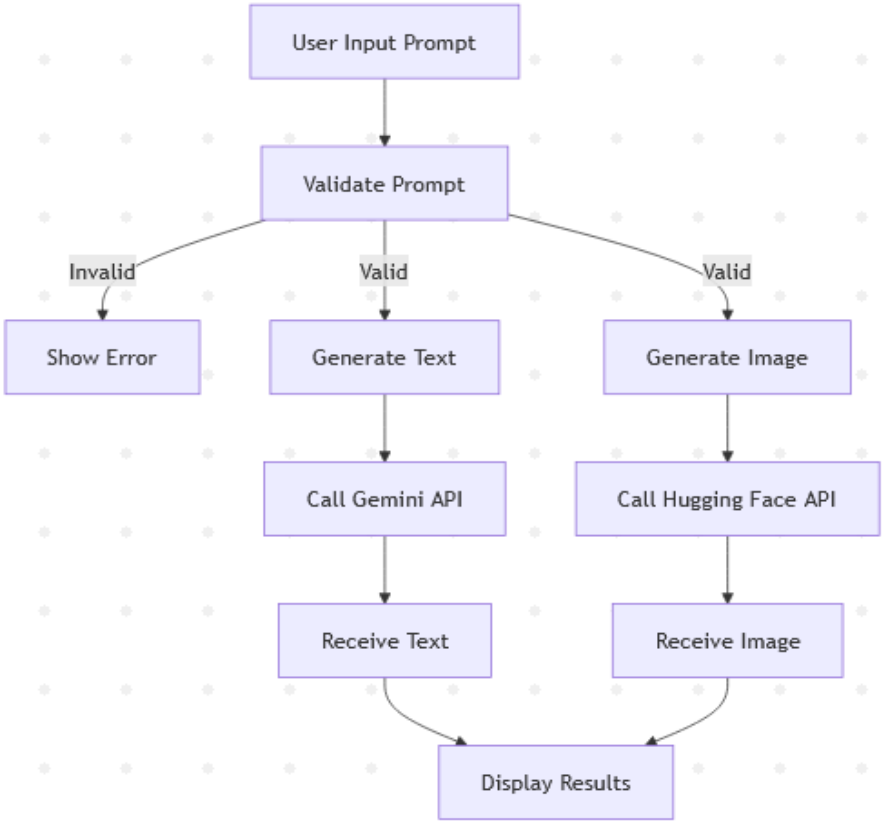
   This design was chosen to keep the demo minimal, understandable, and easy to present.



2.2. Process Flow :

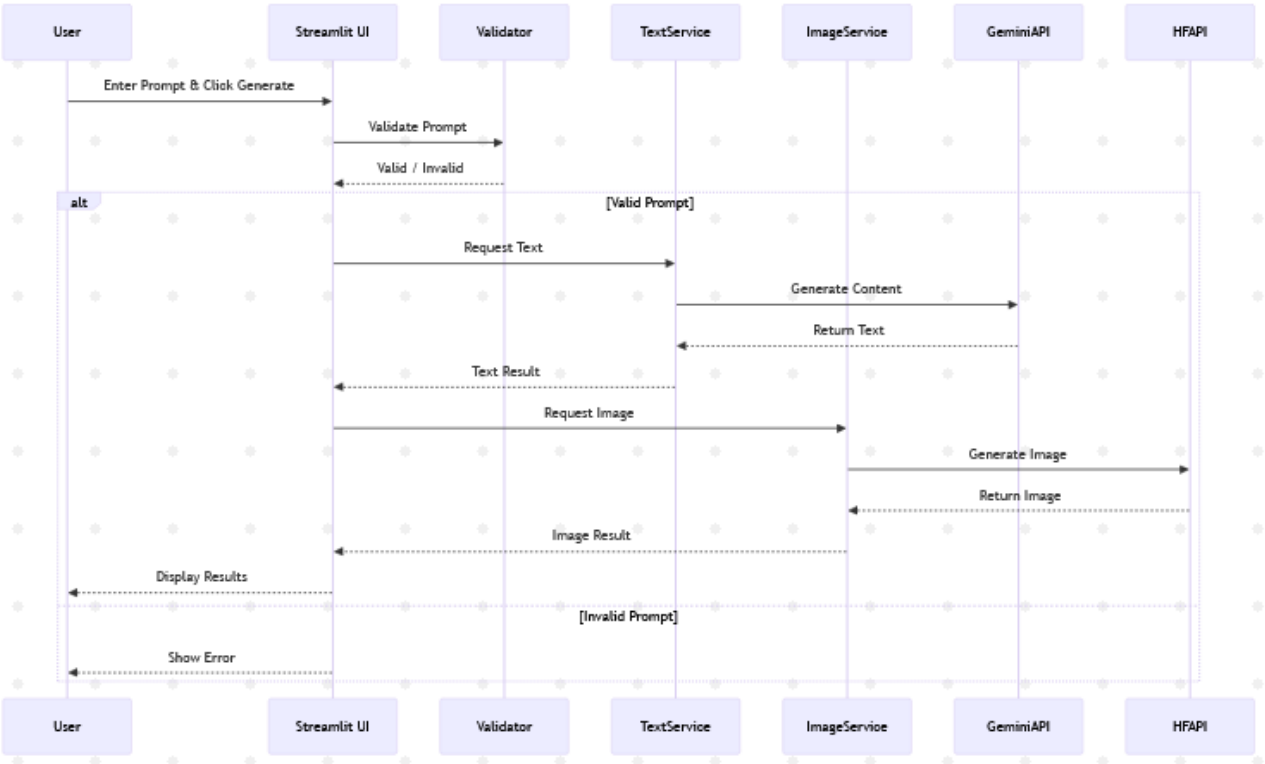
1. User opens Streamlit app in browser.
2. User enters a hospitality concept prompt.
3. User clicks `Generate`.
4. Application validates the prompt.
5. Application sends prompt to Gemini API.
6. Gemini returns descriptive text.
7. Application sends same prompt to Hugging Face image API.

8. Hugging Face returns image bytes.
9. Application renders text and image in the UI.
10. If any issue occurs, the app displays an error message.



2.3. Information Flow :

1. Input source: User prompt entered in Streamlit UI
2. Outbound API call 1: Prompt to Gemini text model
3. Outbound API call 2: Prompt to Hugging Face Stable Diffusion model
4. Inbound response 1: Generated descriptive text
5. Inbound response 2: Generated image content
6. Output destination: Browser UI rendered by Streamlit



2.4. **Components Design** : **Component 1: Streamlit UI**

1. Collects prompt input
2. Displays generate button
3. Shows spinner during processing
4. Displays text output
5. Displays image output
6. Displays error messages

Component 2: Gemini Text Generation

1. Reads `GEMINI_API_KEY` from environment
2. Creates Gemini client using `from google import genai`
3. Calls `client.models.generate_content(...)`
4. Returns generated text as string

Component 3: Hugging Face Image Generation

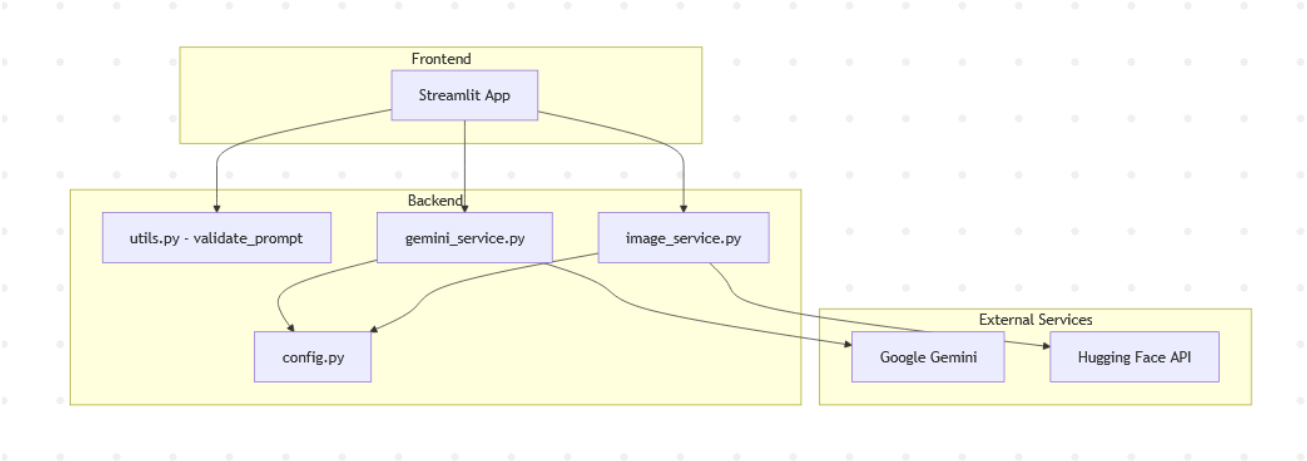
1. Reads `HUGGINGFACE_API_KEY` from environment
2. Sends HTTP POST request using `requests`
3. Uses Stable Diffusion model endpoint
4. Returns image bytes for display

Component 4: Configuration and Utilities

1. Stores model names and endpoint constants
2. Validates user input before making API calls

Component 5: External API Services

1. Gemini handles text generation
2. Hugging Face handles image generation



2.5. Key Design Considerations :

1. Simplicity: small modular project for quick understanding
2. Demonstrability: focused on visible text and image output
3. Minimal modularity: UI, service logic, config, and validation are separated
4. Maintainability: short and readable code
5. API-based integration: no local model hosting required
6. Local deployment: suitable for classroom or presentation demo

2.6. **API Catalogue :**

Google Gemini API

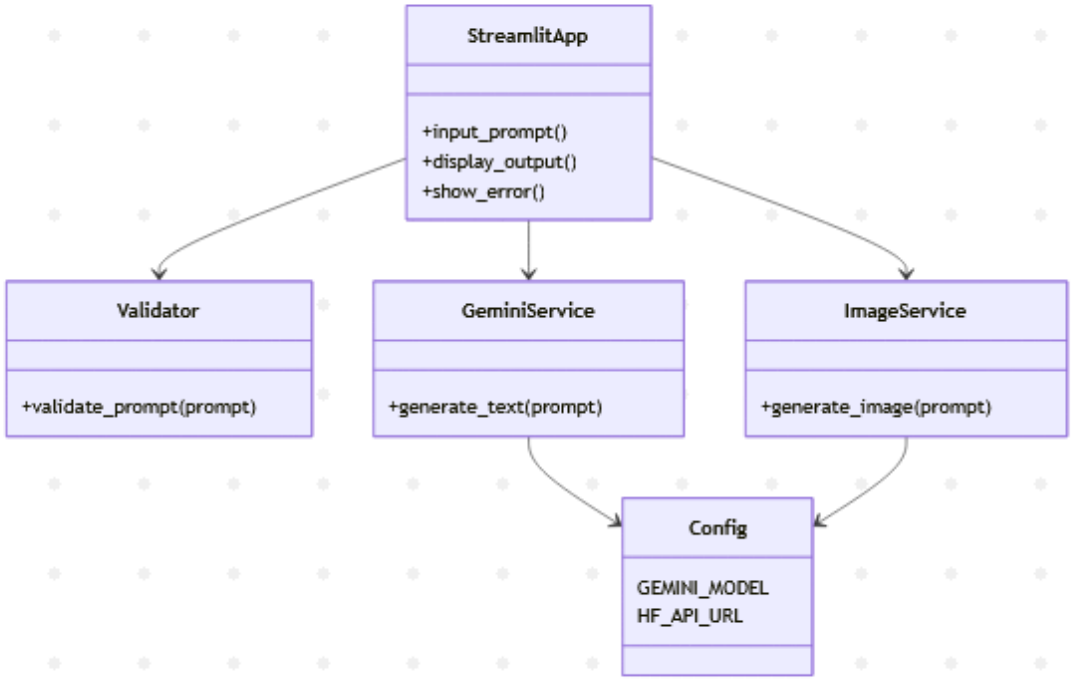
1. Purpose: Generate hospitality concept description
2. SDK: `google-genai`
3. Method used: `client.models.generate_content`
4. Model: `gemini-2.5-flash`
5. Authentication: `GEMINI_API_KEY`

Hugging Face Inference API

1. Purpose: Generate hospitality concept image
2. Access method: HTTPS POST request

3. Endpoint style: Hugging Face router inference endpoint
4. Model: `stabilityai/stable-diffusion-xl-base-1.0`
5. Authentication: `HUGGINGFACE_API_KEY`

2.7. **Class Diagram :**



3. Data Design.

3.1. Data Model :

This demo does not use a persistent database. The main data objects are transient runtime values:

a. `prompt`: input text from user
b. `text_result`: generated text response
c. `image_result`: generated image bytes

3.2. Data Access Mechanism :

Prompt is collected from Streamlit input widget

a. External AI services are accessed through SDK/HTTP APIs
b. Returned data is used in memory only during request processing

3.3. Data Retention Policies :

No local persistence of prompts or outputs by default

a. Data remains in process memory only for the active request/session
b. Logs may exist only if manually enabled during local execution

3.4. Data Migration :

No data migration is required because the application has no structured persistent storage.

4. Interfaces

The system exposes a browser-based user interface through Streamlit.

User interface elements:

1. Title banner
2. Prompt input box
3. Generate button
4. Generated text section
5. Generated image section
6. Error message display External interfaces:

1. Gemini API interface
2. Hugging Face inference interface

5. State and Session Management

1. Streamlit manages per-session UI state during the browser session
2. The app does not maintain custom long-term session state
3. Each generation request is processed independently

6. Caching

1. No explicit caching is implemented in the current demo
2. This keeps behavior simple and transparent for presentation
3. Future enhancement could cache repeated prompts or generated results

7. Non-Functional Requirements

7.1. Security Aspects :

1. API keys are stored in environment variables, not hardcoded in code for normal use
2. Sensitive credentials should not be committed to version control
3. HTTPS is used for communication with external APIs
4. The demo should be used only with controlled access in local development or classroom presentation setup

Recommended improvements for production:

1. Secret management tool or `.env` file with secure handling
2. Role-based access control
3. Request logging and monitoring
4. API usage throttling

7.2. Performance Aspects :

1. Application performance depends mainly on external API latency
2. Gemini text generation is typically faster than image generation
3. Hugging Face image generation may take longer depending on model availability and queue time
4. The UI includes a spinner to indicate active processing Potential future improvements:

1. Parallel API invocation
2. Response caching
3. Retry logic with backoff
4. Async processing for better responsiveness

8. References

1. Streamlit documentation
2. Google Gemini API Python SDK documentation
3. Hugging Face Inference API documentation
4. Python `requests` documentation